

Table of Contents

Abstract.....	3
Market Research	3
Operating systems.....	3
Architectures	4
Devices.....	4
ICMS Surface Level Architecture.....	5
ICMS Alternatives.....	5
Home Assistant.....	5
IFTTT	5
Comparison	5
Product requirements	5
Expected userbase	6
Prototyping.....	6
Lo-Fi Interface Designs	6
Figma Designs.....	7
Agile Development.....	9
Expected Development Risks.....	10
Unexpected Development Risks.....	10
Git/GitHub	10
Setting the Project Foundation	10
Initial Interactivity	11
Logging and SSL Support	11
Extensions	11
Community Store.....	11
Social implications	11
Legal requirements and protecting users and their data.....	12
Software Architecture	13
Connection methods.....	13
Webapp or Native Application?	14
Svelte/Sveltekit.....	14
Tailwind/Daisy UI	14
Other Languages	15
Python	15
The Python GIL	16

MQTT and Alternative Server Protocols	16
Sockets and web-sockets	16
Server Functions	18
Comparing Libraries.....	18
Django	18
Fast API.....	18
Flask.....	19
Authentication	19
JWT Tokens.....	19
Caching	19
Database/SQLite	20
Extending the application	20
Extensions	20
Testing and Analysis	21
Usability Testing	21
Feedback	22
Ease of development vs best practices.....	22
Development Environment	22
Hosting and improving on ease of use	22
Docker	22
Docker-Compose	23
Nginx	23
Optimising The Application/Future development.....	23
Improving the store.....	23
Improved extension management	23
Problems with development	23
What I've Learned and Improved During the Project.....	24
Final thoughts and discussion	24
References	24

Abstract

IoT (internet of things) is a method of describing a network of embedded devices. These devices are often low power and utilise a connection such as Wi-Fi or Bluetooth to exchange data with other systems sometimes over the internet.

As a population we have become increasingly more and more dependent on IoT devices while going about our day to day lives and as a result these devices can be found everywhere. From things such as controlling the temperature of our homes, to controlling our lights and how we interact with the internet using devices such as Amazon Alexa or Google Home, or even things down to smart fridges and ovens. Many of these devices are designed with human interaction in-mind and as such are designed to ease the lives of the owners.

Traditional iot devices use small integrated boards lacking operating systems like the ones found on drastically more powerful systems such as pcs. This lack of traditional operating system comes with unique challenges such as how to interact, program and operate the device.

This report will cover how these challenges have been faced throughout the design and how the system produced will tackle them.

Market Research

The market for these devices has exploded while the tools and ability to develop for these devices has been attempting to catch up.

In the past creating new low power devices was far more difficult than now. Technology has now advanced to a point where low power devices are massively more accessible to the average hobbyist or developer as they are so much cheaper. Devices such as raspberry pi's and esp32's can be ordered for as little as £10 with access to Bluetooth and WIFI that rival that of mobile devices such as phones and laptops for dramatically less of a cost.

As different devices have been released the communities around them have also continued to be built. More specifically, more hardware such as screens, adapter boards have been designed and built for cheaper and cheaper prices. Because of these prices, hardware has become more split between devices as different manufacturers develop their own unique software and hardware.

Operating systems

Two of the key things differentiating ultra-low power devices from other traditional computers is their operating systems and system architectures.

The operating system is the piece of software that runs all other software. Low power devices tend to utilise a very minimal operating system, as a result, they lack luxuries found on other more powerful devices such as graphics/GUIs and support for many of the ports expected by modern standards such as multiple USB ports or output ports such as HDMI or DisplayPort. As a result, building software for these devices is significantly more difficult and developers have been required to find entirely new ways of thinking when it comes to them.

Architectures

Many low power devices utilise an ARM SoC (System on a Chip), ARM takes advantage of a reduced instruction set in order to be far more optimised than other CPU architectures for efficiency and simplicity. As a result, by utilising these SoC's the devices can stay powered for significantly longer, with a lot less system overhead from the more complex architectures. Another advantage of using these SoC's is their ability to integrate other hardware within the system. For example, the addition of Wi-Fi and Bluetooth capabilities can be integrated into one small package reducing its footprint allowing for it to be used on increasingly smaller devices. The ability for wireless communication is especially important for these types of smaller devices.

Devices

The Raspberry Pi is one of the most well-known examples of low power but powerful devices. Raspberry Pi's are often chosen due to their community support and ease of development.

Devices such as the Raspberry Pi Pico or Zero are examples of their extra low power range. These devices utilise relatively powerful soc's while reducing their total power usage to significantly lower amounts than other devices such as the full-size Raspberry Pi's and other small form factor PC's.

The Raspberry Pi Pico W was a significant inspiration for many of the design decisions made within ICMS. The W in the name signifying the fact it has a Wi-Fi chip built in, allowing the device to send and receive data over the internet. This ability to communicate wirelessly makes it a perfect candidate for being used to create a Wi-Fi enabled IOT device. When using the Pico or other Raspberry Pi devices a huge benefit is the fact that the devices can be developed using the Python programming language.

Like with the PC's version of python, many different packages and libraries can be utilised when building for these devices. Python is just one of the many languages that can be used to build these devices, a few examples of languages that can be used are:

- C++
- Python
- Rust

One of the aims for the ICMS store is to bridge the gap between the developer and software by making it available to everyone. This includes different software written in any languages for many different devices.

ICMS Surface Level Architecture

ICMS is a set of applications to make developing, debugging and hosting IOT devices on a local or cloud system, as simple as possible.

The ICMS server is the main connection for the devices. This server hosts all the extensions as well as the socket server where the devices will connect.

The ICMS app is a svelte web app that is designed to be used as a dashboard to control the server. The web app allows you to define users and their individual roles on the network, as well as pair and control each device.

More features can be added or removed by using the extension system. The extension system is a way of allowing community created code to be run on the system by simply downloading and installing libraries in the extension folder of the server. These extensions can range from simple things such as being able to control Spotify, or the implementation of more complex systems such as full neural networks/AI systems.

ICMS Alternatives

Home Assistant

Home assistant is an open-source software also written in python. It allows the user to automate and connect devices to a central server similarly to ICMS. The main differences between the two pieces of software are that ICMS is designed for developers or hobbyists to design and build their own devices, while home assistant is designed for connecting already built devices with software designed by the manufacturer.

IFTTT

IFTTT is a closed-source software similar to home assistant. The applications of this software are quite different as it can be used on any device, for example, being used to send notifications to a mobile device. One of the issues with this system is that it is closed source, where development of the software is up to the organisation's discretion and has no direct community access. Instead, automations have to be built through their no-code system and integrated through their self-hosted store.

Comparison

[Show comparison here](#)

Product requirements

There will be a server that will allow different devices to connect and authenticate. This server will have the ability to store and reference different devices based on the current user selected per device.

Expected userbase

The expected userbase of ICMS is hobbyists. The application gives them the ability to host it on their own local networks such as a home lab or server and be able to control and develop for their own devices. For example, simplifying development for things such as the Raspberry Pi or smaller devices such as ESP32's or Raspberry Pi Pico's.

The developers using the software are expected to be able to program in the python programming language as this is the language selected for use when developing different extensions.

Prototyping

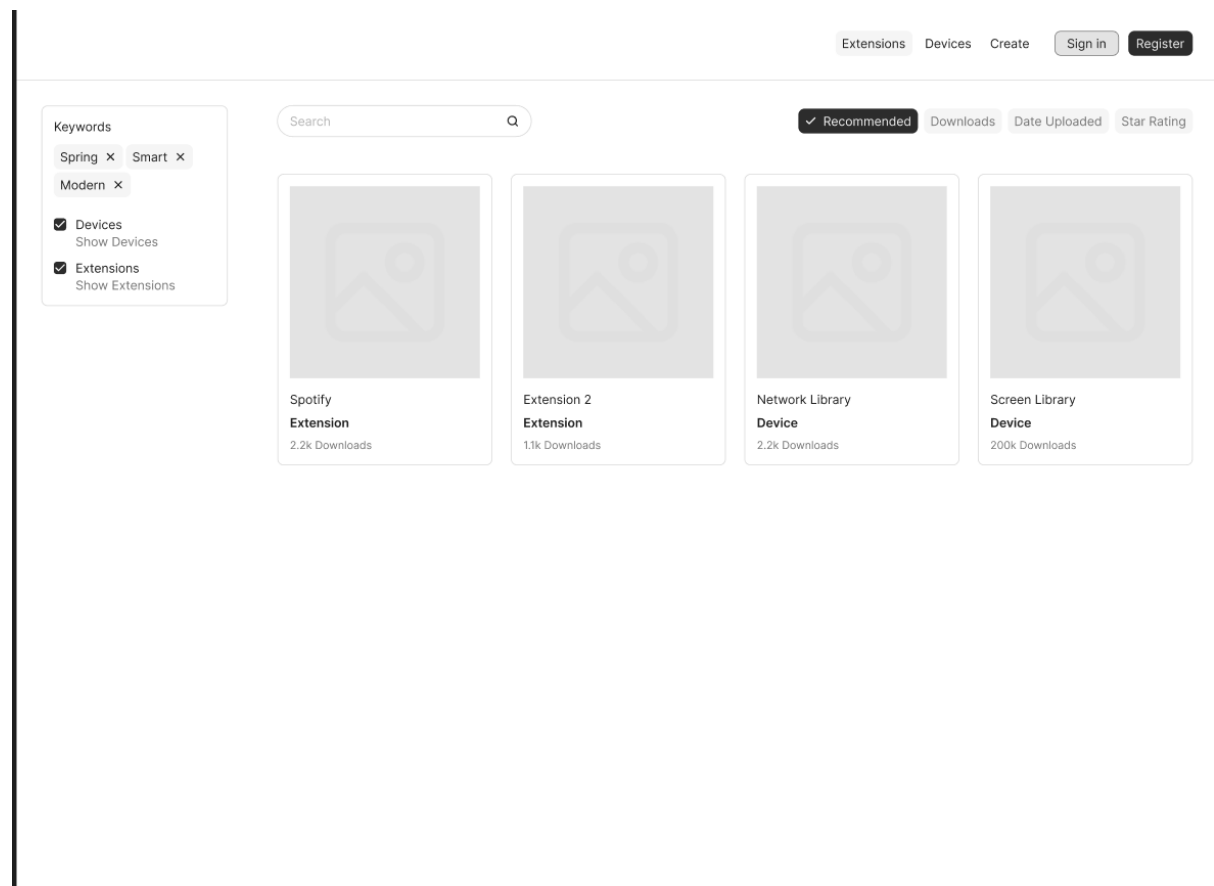
Lo-Fi Interface Designs

Figma Designs

Figma was used to get a basic understanding of how the web app were going to be laid out when being developed. Figma allows the developer to use different UI libraries that look similar to components found around the web. Using these libraries, web pages can be built to emulate the look and feel of a real web page without all the code that comes with it.

The ICMS Store was the first to be planned out using Figma:

<https://www.figma.com/proto/hIRHGEGuFUIP2NclzqttzJ/ICMS-Store?node-id=0-1&t=zB6URC4gdtq2d0cH-1>



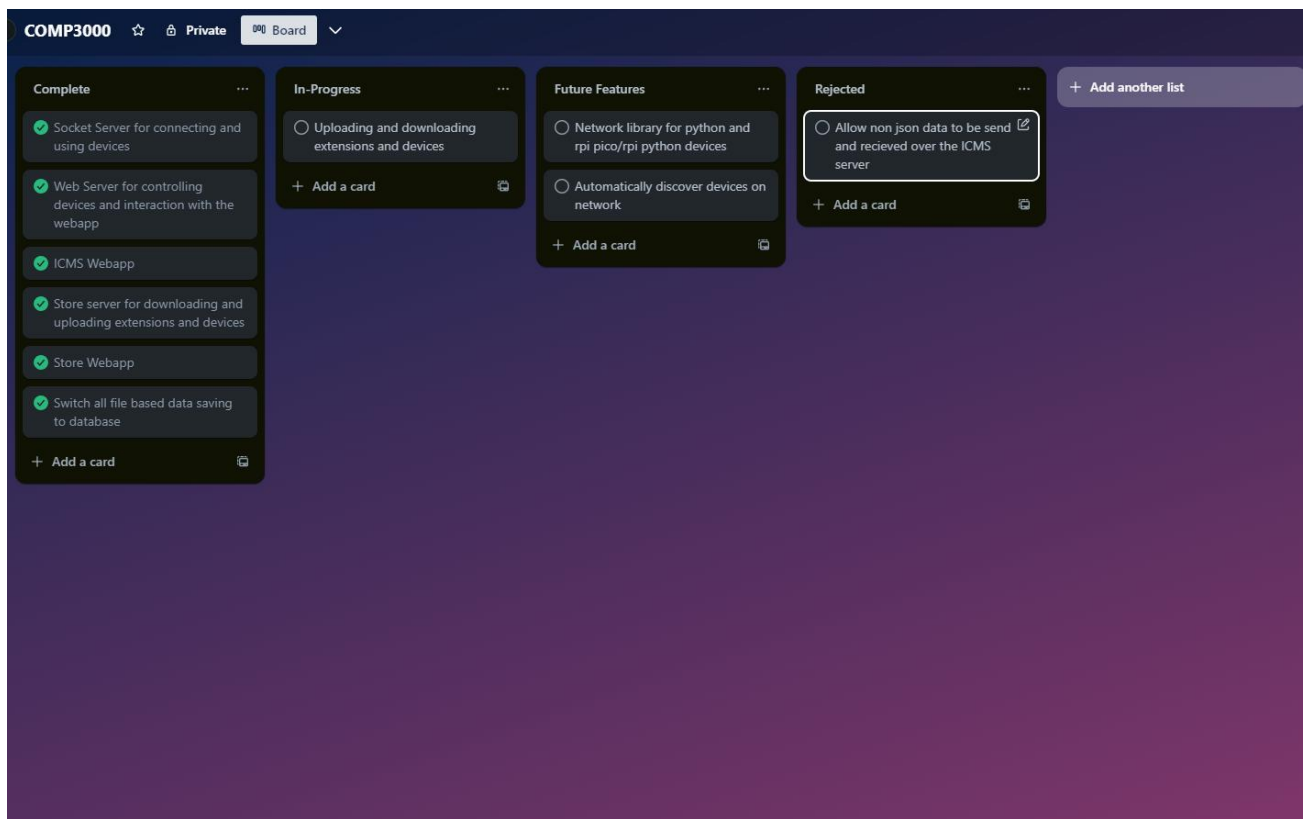
This main store page was designed to be as easy to navigate as possible for all users. Including many features such as, the ability to search and sort devices or extensions based on keywords and their different types.

(icms server figma)

Agile Development

The development of the project followed agile principles of development. Tasks were first written and planned out, the chosen software to track this was Trello. Trello is an easy to use and intuitive piece of software that allows users to specify lists of tasks which can be marked when completed, more advanced features can be utilised such as setting up automations, where unique functions can be carried out based on rules that have been set.

<https://trello.com/b/Q10qkCf9/comp3000>



Following the tasks being planned, development can begin.

Development is then followed by testing, which was carried out on a per application basis (see. [Testing and Analysis](#))

One of the key parts of agile development is the review stage. Feedback was gathered through meetings with the project manager. Each meeting was used for feedback on the project as it progressed.

Receiving that feedback allowed the project to progress significantly faster (iteration).

Expected Development Risks

Some expected risks when creating a project of this calibre

(not being fully developed at the end)

(over time)

Unexpected Development Risks

Creating a project like this also comes with some unexpected risks to development.

(loss of data)

(unforeseen absence / illness)

Git/GitHub

Git was the chosen method of version control throughout the development of the application.

Setting the Project Foundation

<https://github.com/SSkimmd/ICMS/commit/e0bee6f322032a80729417e9a2b7740624d5eb3a>

The initial commit for the project only included the primary socket server. The thought process behind how the application was originally going to work was that a separate monitoring service would run with a reference to the socket server where it is able to manipulate it. This is similar to the end product although instead of being multiple services, it was instead decided to include both servers in one application utilising different threads to minimise the number of services that would need to be run.

Another feature unique to the first few commits that was dropped from future iterations is the capacity for the socket server to process http requests in combination with the raw byte data. It used a function to detect and parse the request and return an extremely basic http response with manually appended headers. This change was made when the flask web server was added as the functionality was unused in most cases.

An initial version of the webapp is also included in this version of the project but is lacking any functionality or user interfaces. This initial commit was overall used to setup the foundation of the project and determine the technologies that would be used.

Initial Interactivity

<https://github.com/SSkimmd/ICMS/commit/75dbee7814afa73d8baf6d5b22c94075626801c0>

This is the next significant commit as it contains the initial implementation of the ability for the webapp to interact with both servers and devices.

This commit is where the initial design theory for the webapp was set with the inclusion of Tailwind and Daisy UI in the svelte app.

This version of the application utilised the different way of defining Flask Api endpoints, where the endpoints are not defined within a class. It also lacks the async/await architecture found in the later versions.

Logging and SSL Support

<https://github.com/SSkimmd/ICMS/commit/ff96ec68ab6003448b723bd62b10a1ce8d1eeb33>

One of the key features of ICMS is the ability to debug and develop for different devices. With the addition of logging support, the developer has the ability to view any errors in real-time through their dashboard in the webapp.

Another big feature of this version of the project is the inclusion of SSL/TLS support for the socket server. The SSL context used by the server requires the developer to specify two certificates found in the certificates folder before integrating with the server. This feature is designed for developers who don't require the use of a nginx proxy nor access through the web app and would instead prefer to run only the socket server on their own protected network.

Extensions

<https://github.com/SSkimmd/ICMS/commit/54c77071bdb049eca7d8b420b7a1d4549f3482de>

This commit is the first integration in the project of an external Api being called from an extension. The Spotify extension was introduced in order to test how interactions between the servers and other Api's could work and was integrated seamlessly, utilising Spotipy as the library used to interact with the Spotify Api.

Community Store

The community store is another addition to ICMS that allows users to upload and download different extensions. The store is also used as a hardware database, this allows users to create and upload the software for specific devices using specific hardware. This is in an effort to prevent gatekeeping in the community through the open sourcing of all the software uploaded.

Social implications

Opening the community store brings a large number of social implications for the application. One of the biggest benefits being that having a large community will massively improve the software ecosystem around the application, giving access to users and developers of any level.

Building a community does come with some negatives though, one significant one being the extra requirement for the moderation of content being uploaded to the store. Software can only do so much with blocking and censoring images and words. Human interaction will be necessary if the community is to survive and thrive. Moderation can also be extremely difficult depending on the content, for example, determining and managing copyright or stolen content becomes increasingly difficult when more and more claims are made especially when having to decide who the original owners are.

ICMS and the ICMS store will also need to store and access data used by the application to determine users. This data can be things such as IP addresses, upload dates, names and email addresses. Careful considerations for how the data's stored and managed will need to be considered to keep the data safe from harm. For example, data stored in the UK will need to follow all UK GDPR principles, while also following all principles and regulations outside of the UK with things such as EU GDPR.

Legal requirements and protecting users and their data

Protecting users is a very important step when developing any system that can store and process user data. Malicious users have many methods of attack, for example, one of the more notorious methods of attack is SQL Injection.

SQL Injections are a method of attack where a malicious user will attempt to take advantage of unsanitized inputs in order to send malicious commands to the database. Unsanitized inputs are inputs such as buttons, search bars, or text boxes that have not been checked for malicious code or words. These malicious commands can come in the form of extra text appended to the end of a string, terminating the previous command and allowing a new command to take its place, essentially giving full access of the database to the user. In order to protect against this type of attack ICMS takes advantage of python's incredibly well supported inbuilt SQLite package to sanitize any inputs before committing a command.

(show code here)

Other attacks are far less obvious than the previously stated attack and can occur without the user ever even realising. CSRF (cross-site request forgery) and XSS (cross-site scripting) attacks are two attacks that need to be understood and protected against when writing applications that are accessible through the web. A CSRF attack is where malicious requests from an attack vector such as a fake website, are made to a legitimate website pretending to be the authenticated user. XSS attacks are similar but instead inject malicious code, through utilising vulnerabilities in the webapp, directly into the user's application. Svelte/Sveltekit being as modern of a framework as it is, has built in protections for both of these types of attacks and those protections are utilised within both ICMS web applications.

Another method of keeping user data safe and secure is by encrypting it. All passwords stored within the application's database are encrypted to ensure that any attempt at attacking the application will result in never having direct access to plaintext versions of them. This method of securing passwords forces the attacker to utilise brute force attacks that attempt to guess the password which are far, far slower.

Software Architecture

ICMS was designed with extensibility as priority one and as a result having the ability to add and remove different pieces of the software through extensions required a very modular design.

Connection methods

The biggest challenge found when designing this project initially was deciding how devices would be able to connect to the server. This would need to be one of the biggest focal points for the application as it was one of the standout features of the entire project.

Wi-Fi is one of the most widely used methods of connecting and using an IOT device. This is most due to the fact that most devices within a household such as mobile devices, PC's, gaming consoles and routers all use Wi-Fi and as a result, the average household is more likely to have the hardware.

Examples of IOT devices that also utilise this connection method are home assistants such as Amazon Alexa and Google Home.

Bluetooth is another method of connecting devices that has slowly phased out of relevance due to security issues and range limitations. Bluetooth can still be found being used for less secure devices due to the ease of connection, for example, in Bluetooth headphones.

Zigbee is a relatively new protocol for IOT devices that has begun to gain more and more traction in the past few years. The main drawback of using the Zigbee technology is the requirement to use proprietary hardware to convert back and forwards between different protocols. As a result, this can cause projects to become significantly more expensive than using the traditional Wi-Fi hardware.

One of the biggest concerns that dictate the type of communication being used by these devices is reliability. The reliability of communication is especially important when a device is used to communicate very important data, which, on delivery needs to be complete and without error. Methods of ensuring reliability change depending on how the data is transported.

Two examples of transport protocols are:

- TCP. This transport protocol guarantees data delivery and integrity by utilising intense error-checking upon delivery. Another advantage of the TCP protocol is that it can also take advantage of retransmission, where the packet of data is resent if the packet fails to arrive.
- UDP. This transport protocol is unique in the fact it does not require a pre-configured connection. Data can be sent and received at any time from any device. Another advantage of using UDP is that the data sent doesn't need to be complete. Instead, incomplete data can be sent even if packets have been lost, unlike TCP.

Both protocols come with their own unique use cases and are designed to be different intentionally.

(TCP use case)

(UDP use case)

Webapp or Native Application?

The type of application utilised by the project is a web application. The choice to use a web application was made for a few reasons, one of the main ones being the fact a web application is cross-platform and has the unique ability for it to be accessed from anywhere on the network it is being hosted on. Hosting an application on the network instead of being a native application allows the user to access it from any device as long as the device has access to the network it is being hosted on. For example, with the ICMS application, the user can access it from a mobile device, or from any other devices connected over the internet. This prevents a system that is specifically built for only one platform, like a traditional windows or mac application would otherwise be. Although the application is by default cross-platform, this will not prevent the application from being used with a system such as electron to be run similarly to a traditional native application instead, but this is left up to the user's preference.

Svelte/Sveltekit

Svelte is a relatively new web framework, building off of what has been learned with other frameworks such as a React it has many modern JavaScript and web features.

For example, one of the key features is their implementation of server-side rendering (SSR). Server-side rendering allows the server to render the web page before sending it to be displayed on the client. In effect, it becomes a read-only system where the functions and data retrieval all occur on the server preventing the client from having direct access to them. This architecture allows the client and server to both be protected from attacks from malicious users.

Some issues do come from utilising this architecture though. Data stored on the webpage is still accessible to client, meaning data still needs to be protected and only retrieved when necessary. Another difficulty when using SSR is client interactions such as submitting forms or data from the front end to the backend. To get around this, Svelte uses a system called server actions. Server actions are exported functions which the client has access to, which allow the front-end to interact with the server. These functions aren't directly accessible from the front-end but instead are submitted similarly to an http endpoint through forms or fetch requests.

(show some examples here)

When submitting a form, all form data is included within the request.

Tailwind/Daisy UI

Tailwind is a CSS framework used to easily modify how different html elements are displayed directly in their definitions, this removes the need for large amounts of CSS files littered around the application. This way of defining element parameters in a simple form is an incredible resource for developers that aren't as graphic design centric.

Although tailwind comes with many benefits, one pretty significant downside to tailwind is how complicated some elements can become. Defining CSS classes directly in the elements can cause them to become extremely bloated and hard to read.

Daisy UI is a tailwind CSS plugin that comes with pre-made components that can be used as drop-in replacements for the default html elements. All that is necessary to use a component is including its name in the class definition of the element chosen.

Other Languages

Many languages were trialled for use in ICMS. One of the key ones being the GO programming language. GO was initially selected for its potential speed and ease of development, heavily benefiting from an almost python-like ecosystem and community with a large number of libraries and incredible levels of support.

One of the main deciding factors against selecting GO was the necessity for importing and using different extensions, the system would need to be manually built from the ground up if supported at all. Another significant deciding factor was lack of knowledge of the entire language and building system. The amount of time that would be necessary to learn it to a level for such a project would out-weigh the total time for the project. As a result, GO was not chosen for the project and instead Python was selected.

Python

Python was the chosen language for the development of this project because of previous knowledge of the language, ease of development and the community support.

Community support was especially important for the project as the project was designed around also having the community develop extensions for the application. Using a well-known and well supported language allows more people that would other not be able to contribute to be given the chance to, as well being able to utilise different libraries and packages to simplify the development without having re-invent the wheel.

Python is an incredibly versatile language. A part of this versatility stems from the fact it is an interpreted language versus being a compiled language. Being interpreted means the code is run without first being compiled, this means the code is run line by line executing each command as it comes. This massively affects the development experience as skipping the compilation step allows for much faster development, this is also why it is often chosen for web-development where hot-reloading and quick iteration is far more common.

A downside to being interpreted is speed. Python is not widely regarded as being the most efficient or fast language out there, it misses things such as compile-time optimisations and has a large amount of overhead in comparison to language such as C/C++ or Java/C#. To attempt to combat this, python utilises compiled C libraries to replace some of the slowest parts, while also using a caching system to compile some parts of the application to python byte code which is much closer to native code than running the interpreted code.

For most applications, the speed of python is more than enough. Issues with speed begin to appear when projects scale quickly with a lack of optimisation. Before a lengthy re-write, there are many methods of increasing speed without necessarily having to write any new code. For example, things such as separating the systems of the project so that multiple instances can be run with their own resources. Occasionally though, large scale projects can outgrow the speed available from the language.

Modern versions of python include the keywords for async await execution. Async architecture allows for the concurrent execution of code, meaning, instead of having to wait for code to execute other code can instead be executed and processed while waiting. All server functions in the ICMS server are built using the async architecture because extensions are designed to be able to run in the background depending on what needs to be processed, through the asyncio

python library. This design also helps when utilising different threads for things such as, writing or reading to or from a device multiple time a second.

The Python GIL

The python GIL protects from cross-thread errors such as race conditions by detecting when the application attempts to access objects unsafely. Many languages leave this up to the developer to protect against, most notably C/C++ has no protection against thread unsafe code and as a result many errors can occur when attempting to write multithreaded code. Through the GIL python instead does this automatically.

MQTT and Alternative Server Protocols

There are multiple different protocols designed specifically for IoT devices that could have been used within this project. For example, MQTT which is an OASIS standard messaging protocol for the Internet of Things (IoT). (MQTT, 2024)

MQTT is designed to be lightweight, reliable and allows for the integration of security features such as TLS/SSL encryption. The protocol uses a publish-subscribe pattern, where data is published to a topic, similarly to that of socket-io rooms, and devices subscribe to the topic in order to receive and view the data from that specific topic. This pattern allows for real-time updates as the end user will receive an update when messages are sent to the subscribed topics.

The choice to not use these protocols was made to ensure all of the design choices were made in-house. This allows ICMS to take advantage of the ability to fine tune every step of the process, from the sending to the receiving.

Sockets and web-sockets

The ICMS application is comprised of two different servers, a flask http server and a socket server. The key differences between these two servers are the HTTP server uses the REST architecture for receiving HTTP requests, where each request can only be one type of HTTP request, for example, a GET or POST request. The issue with a HTTP server is that data can only be transferred when a request is made, meaning streaming data between the server and device can only occur if the device either makes many requests to the server, or similarly, the server makes many requests to the device. In the case that the latter method was chosen, the device would also need to be running its own HTTP server, which would also be introducing a performance over-head of which on many of the potentially extremely lower power devices, needed to be avoided.

Instead, the socket server is the server used to communicate with the IoT devices. The socket server instead reads and writes from a stream of data, where data can be sent and received at any time from either side. This massively reduces the over-head necessary as instead of having to make many requests, the device can instead simply create and keep open one connection to the main server that can be reused to send and receive.

Using the socket server does come with some caveats. Mainly being the unstructured form of data being sent. In comparison to the pre-formed HTTP requests which can be easily read and manipulated, the socket server and clients send raw bytes of data between each-other which need to be decoded and formed into readable data. JSON was the form of data chosen to be used by the server and clients as it is an incredibly versatile data format widely supported by

languages and different libraries and systems, while also being incredibly human read and writable. Python especially has amazing built in support for the JSON format.

(elaborate on how the commands are processed and the types of data sent and how they're determined)

Commands are processed using a type identifier. There are 3 main types of identifiers:

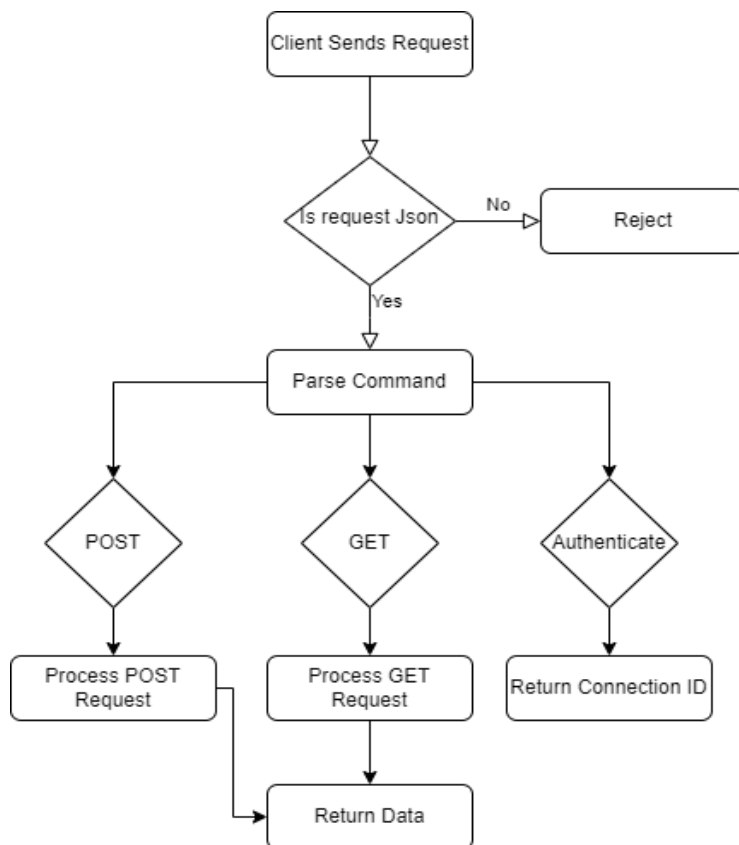
- GET
- POST
- Authenticate

The identifiers are labelled similarly to how a HTTP type would, for ease of recognition. When receiving a GET request the server will process what need to be accessed, then once the data have been processed the data will be returned.

When deciding what needs to be accessed, the server will retrieve the GET part of the request to determine if the type of GET request is a device or extension request. The difference between these types being that a device request will return devices connected to an authenticated account, while the extension request will return whatever the function being called returns. This is very similar to how the POST request works in that there are two types of POST request, device and extension.

The authenticate request is made from device to server in order to pair a device with an authenticated account, passing a username and password in order to login to the account and pair. The connection-id of the device is then returned in order to be able to send requests from the device.

Server/Client Diagram



Server Functions

When connecting a new device to the socket server, the device has the option of specifying a list of endpoints. The endpoints are functions which are the way of displaying to the server and any other device, what features the device has access to. This is similar to architectures such as remote procedure calls (RPC) where functions can be directly called, and the arguments are determined by the request.

(elaborate here)

Through the extensions, new features can be added to the server, automatically updating what new functions devices have access to.

Comparing Libraries

There are many different systems for hosting HTTP servers with python. 3 key ones stand out above the rest and are relied on by many different organisations for their backend requirements. These 3 systems are Flask, Fast API and Django.

Django

Django is one the most well-known python full-stack libraries for building servers and is by far the most professionally used. Django powers the backends of the likes of Facebook and Spotify.

Fast API

Fast API is another example of a different API for building servers within python. One of the key differences with other API's and Fast API is their abundant use of the Pydantic library. Pydantic

is built to make modelling data within programs easier and is the basis of Fast API's data modelling system.

Flask

Flask is an incredibly minimal library for hosting HTTP servers. Some of the key factors of selecting this library for use in the application was the community support and feature richness. Community support for the library allowed for very rapid development and iteration especially important for web development.

The library supports many of the more modern features found in python, such as the async/await architecture as well as support for modern hosting applications, one of which is gunicorn, the system selected to be the host for the production build of the application.

Building endpoints for flask is incredibly easy, endpoints are determined by decorating different functions or adding them at runtime to the flask application object.

Taking advantage of how simple the endpoints are for flask, extending functionality was made significantly easier in the form of function decorating. Decorator functions are used throughout the project for the integration of the authentication system. The decorators live above the functions and are called just before the function is called. This allows processes to occur such as checking if the user calling the function is authenticated, before allowing them access to it.

(show decorator)

Authentication

Authentication is a large part of any application, deciding who can and can't access it and how the user's data is stored. The ICMS application gives users 2 options, locally storing the user data. Or storing the data in a self-hostable authentication service.

Storing the data locally should be fine for most users as long as the main system is utilising the correct security features of the operating system it is running on. To add a layer of security to the user data, passwords are store in their encrypted forms.

The authentication service is another simple server that runs as an API service. Utilising an SQLite database to store the user data, this method of storing data is much more developer friendly.

In order to prevent having to constantly check passwords, the applications including the store implement a token system. Tokens are session-based methods of ensuring a user is who they say they are per request to the server.

JWT Tokens

A JWT token is an internet standard way for creating and storing encoded data within a token. Within the application, JWT tokens are used to store the userID of a user. The userID can then be used to match and find the relevant data for the user from the database or cache.

Caching

Caching user data speeds up the application by reducing the amount of much slower IO processes that need to occur. An example IO process is having to read from a file every time the user wishes to access it, which is a slow process, especially if it was required to be carried out

for every user. Instead, the ICMS applications retrieve the user data and store it in a list in memory.

Unfortunately, as with all things in software development, utilising this system does come with some drawbacks, namely being that caching systems can be incredibly memory hungry on lower power devices and can cause some issues depending on the size of the total amount of data needing to be stored. Another issue is determining when the user data is not going to be accessed anymore as keeping user data that has been unused for a long time becomes extremely memory inefficient. For applications such as ICMS, it was designed around the total amount of users ideally being a low amount, so it then only utilises the database as a form of a backup for the user data, so that it can be retrieved when the application is shutdown.

If the application were to be scaled to bigger amounts of users, the system would need to be updated to reflect that.

Database/SQLite

The database chosen for this application was SQLite. The decision to use this database was made with the same ideals as the rest of the application, in that it is simple to design and integrate through python. This application also chooses to utilise the added functionality of Async/Await architecture through the aiosqlite library.

SQLite has a very low-level Api where queries are written manually by the developer. These queries utilise string manipulation in order to prevent exploits such as SQL injection (see. [Protecting users and their data](#))

(show code here)

Extending the application

The system for loading extensions in the ICMS server utilises the fact that python has easy access to a global system where the libraries are stored and displayed. Having access to this system means libraries can be imported and unimported at will. It also means that individual libraries can be used similarly to objects where the library object can be stored and referenced.

In the application extensions are loaded before any of the other parts of the software are started through the `init_extensions` function found in the webserver. These extensions are loaded by importing it at runtime through python's `importlib` library. This way of loading the extensions allows the end user to decide which libraries need to be imported and where from using the json file found in the settings folder. This massively simplifies the end user experience for building and using community made extensions.

This system also massively benefits developers as the core systems can continue to run after importing an extension even with broken code as the extension will just fail to be imported through the try-catch statement. This failure to be imported will also display a debugging message in the log file making tracking down errors significantly easier.

Extensions

Each extension can register different functions through the extension class. Registering a function automatically adds it to the list of server functions accessible through the web server or through a device connected to the socket server. When registering a function, the developer

also has the ability to specify different arguments for when the function is called. When calling a function through one of the previously stated methods, the arguments specified in the request will need to conform to the arguments stated when registering the function, for the function to be successfully called, otherwise the function will fail and display a debug message in the log. These arguments are created dynamically through specifying it as a string, meaning when converting the variable to the specified type, as long as the type is loaded within the application, it can essentially be anything, and it will attempt to be converted.

(show example of complex data type and simple data types)

Testing and Analysis

Unit testing was conducted on both apps and servers. Testing of all the applications that make up ICMS was more difficult due to the multiple languages utilised throughout. Each language utilised their platforms best testing frameworks in order ensure they were tested as thoroughly as possible.

Pytest was the chosen frameworks for the python applications. It is a well-supported and well-known framework for writing easily readable tests.

(show tests (after you write them))

Vitest was the testing framework recommended for use with Svelte applications and as a result is the reason it was chosen for this project.

Usability Testing

Another form of testing used throughout the project was usability testing, which was carried out in person in order to get the best results. Each test was setup in a way that each tester was asked to complete a different task and rate their experience 1-5 based on how easy or difficult the tasks were to complete.

The tasks asked to be completed for the IoT management application were:

1. Create an account and login.
2. Access and call a server function.
3. Create and pair a new device to their account.

Another set of usability testing was carried out for the store application. Similarly, the tasks asked of the tester were:

1. Create an account and login.
2. Access and download a new extension or device.
3. Upload a new extension or device.

The results of the ICMS tests are as follows:

Tester	Task 1	Task 2	Task 3
Alex			

At the end of this round of testing the testers were then asked to give written feedback on their experience with both applications. This method of feedback was used to drive the development and improvement of the applications visually and functionally in response to their feedback. The testing for this application could have been significantly improved with an increased sample size, but due to outside limitations could not.

Feedback

(show feedback here)

Ease of development vs best practices

Sometimes in development the ease of development out-weighs the speed of the application. As long as the product is proved to be able to be developed. The application can always be optimised or re-written in a different language to be faster.

As a business, best practices should be followed more closely as the development power is usually there to be able to develop it correctly the first time.

Development Environment

Hosting and improving on ease of use

Hosting a server isn't always a simple job. Automating the more difficult parts massively improves how the end user accesses and uses the application.

Docker

Docker is an application built to containerise other applications. Through this containerisation, developers can build and test their code without having to worry about

Docker was chosen to simplify the build process for both applications. Docker allows the developer to automate the build process by using step-by-step commands in a file called a Dockerfile. These Dockerfiles allow you to specify how libraries are downloaded and installed as well as how the final program is run. To use the Dockerfile, an initial image is selected for usage within the container, for example, each of the application servers running within ICMS use the default python image, while each of the Svelte web applications use the node image.

Commands used within docker are the same as any other. The final command is the one that will attempt to run for as long as the application is running, specifying this anywhere else will prevent the command from being executed.

Docker-Compose

Docker compose allows Dockerfiles to be compiled together and run simultaneously in the same docker image. Through the image, different applications can be disabled and enabled depending on which parts the end user wishes to use.

Nginx

Nginx is used through docker, where the backend services are proxied through a nginx reverse proxy, allowing all commands to flow through nginx before reaching the backend services. This system is good for taking advantage of features such as load balancing and rate limiting.

(show diagram here)

Optimising The Application/Future development

Improving the store

One of the main future features to focus on is the ability for the store to track different sets of hardware combinations. Keeping track of these hardware combinations will allow more specific software to be written for them and reduce the overall need to scour the internet to find information. To do this, a database of hardware combinations, using device ids and other identifying information could be created to store them in and visualised through the store page, allowing users to search for exactly what they are looking for.

Improved extension management

Another feature for future developments of the project could be improved extension management using deep links. This would allow users with the application open to open the link and instantly have it downloaded to their applications.

Problems with development

Following timelines

Many parts of the project were re-written over and over.

What I've Learned and Improved During the Project

This project has been an immense undertaking and one of the biggest take-aways from the project is that many features haven't needed to be cut or modified to fit within the deadline. Conforming to deadlines is always important in big projects in the professional world.

Final thoughts and discussion

The market for these devices and the software written for them is massive,

This application doesn't aim to fix every problem but a select few. With any application being developed, feature creep is inevitable. When designing the project, it was especially important to not let too many features be added because the project was never designed to actually fix all the problems found within the design and development of IOT devices. This application was designed to host and manage code written for a central server and allow different devices connected to it access and call that code.

References

MQTT (OASIS) Available at:

<https://mqtt.org/>

What Is the Internet of Things (ORACLE) Available at:

<https://www.oracle.com/uk/internet-of-things/>

Which Is the Best Python Web Framework: Django, Flask, or FastAPI? (JetBrains) Available at:
<https://blog.jetbrains.com/pycharm/2025/02/django-flask-fastapi>